

AID 728 — Course Project Report

Traffic Rule Violation Detection using Computer Vision and Deep Learning

Project Repository: <https://github.com/kaushiksaiyakkala/Traffic-Violation-Detection>

Kaushiksai Yakkala (BT2024240) Deepak Saai (BT2024138)
Saravanan S B (BT2024254)

May 17, 2026

Abstract

This project presents an end-to-end, fully offline computer vision pipeline optimized for automatic traffic violation detection in Indian road environments. Utilizing state-of-the-art YOLOv11 and YOLOv8 object detection frameworks alongside PaddleOCR, the system accurately detects motorcycles, monitors passenger limits (triple riding), analyzes helmet compliance, localizes vehicle registration plates, and extracts license plate text. The complete architecture is highly modular and designed to operate strictly within local compute constraints (< 250 MB total model footprint), demonstrating robustness against dense urban traffic, varying scales, and complex spatial configurations.

1 Introduction

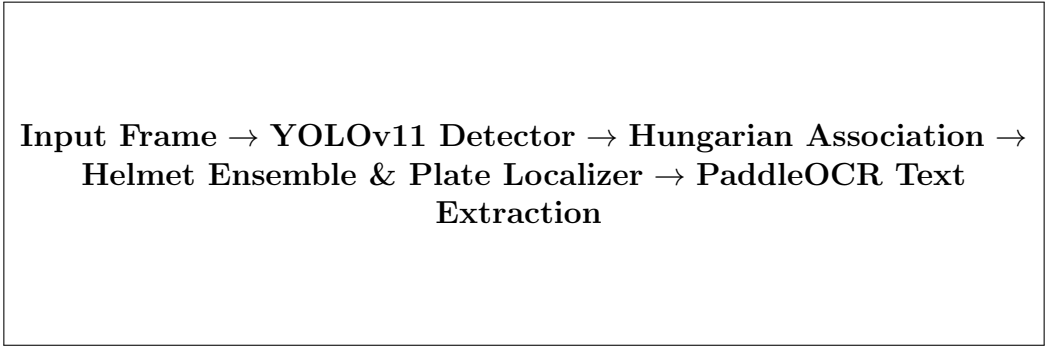
Traffic violations involving two-wheelers are a leading cause of road accidents and fatalities in Indian urban spaces. Common traffic offenses include riding without a protective helmet, carrying more than one pillion rider (triple riding), and operating vehicles with missing or obscured license plates. Manual enforcement of these rules by traffic personnel is heavily labor-intensive, challenging to sustain continuously, and inherently prone to human inconsistencies.

Automated traffic enforcement using computer vision offers a scalable, consistent, and round-the-clock alternative. This project implements a modular, high-performance AI pipeline that processes a single RGB street-scene image to track and flag concurrent violations on two-wheelers. For every vehicle detected committing an infraction, the system automatically isolates the vehicle and extracts its registration number via a specialized OCR pipeline.

2 System Architecture

The system is structured as a tightly decoupled, three-stage pipeline to maximize maintainability and eliminate structural dependencies during debugging:

1. **Detection Stage** — A primary YOLOv11-Large model detects all motorcycles/bicycles and persons in the input image frame. A spatial post-processing aspect-ratio filter automatically handles and discards auto-rickshaws that are occasionally misclassified as motorcycles.
2. **Association Stage** — A Hungarian matching algorithm formulates a bipartite graph to assign detected persons to their respective motorcycles based on global spatial overlap and geometric proximity.
3. **Helmet & OCR Stage** — An ensemble of two specialized helmet detection models analyzes each rider’s head region using proximity-weighted scoring. Simultaneously, a fine-tuned YOLOv11-Nano license plate detector crops the registration plate region, passing it to an offline PaddleOCR model for alphanumeric character extraction.



```
graph LR; A[Input Frame] --> B[YOLOv11 Detector]; B --> C[Hungarian Association]; C --> D[Helmet Ensemble & Plate Localizer]; D --> E[PaddleOCR Text Extraction];
```

**Input Frame → YOLOv11 Detector → Hungarian Association →
Helmet Ensemble & Plate Localizer → PaddleOCR Text
Extraction**

Figure 1: End-to-end multi-stage pipeline architecture

3 Models and Training

3.1 Two-Wheeler and Person Detection

We use **YOLOv11-Large** (`yolo11l.pt`), pretrained on the MS COCO dataset, for isolating vehicles and individuals. The pipeline specifically filters COCO class IDs: 0 (person), 1 (bicycle), and 3 (motorcycle).

YOLOv11-Large was chosen over its smaller counterparts (such as YOLOv11-Nano) due to its high localization capability:

- A significantly higher mAP50-95 of 53.3% compared to 39.4% for the nano version on the COCO benchmark.
- Exceptional localization of heavily occluded riders and distant targets in high-density Indian traffic.
- A compact file size of only 49 MB, which fits comfortably within our strict 250 MB cumulative system constraint.

Images wider than 1280 px are scaled down uniformly prior to model inference to normalize bounding box percentages. Conversely, small images (< 400 px wide) are upscaled to at least 640 px to maximize the feature maps generated for distant two-wheelers.

3.2 Helmet Detection

To maximize inference recall, we employ an **ensemble of two models** evaluated in parallel across the full frame:

Model Identifier	Architecture	File Size	Training Source
<code>helmet_github.pt</code>	YOLOv8-Large	83.6 MB	Fine-tuned on 7,180 images
<code>helmet_iamtsr.pt</code>	YOLOv8-Nano	5.9 MB	Pretrained complementary model

Table 1: Helmet detection models used in the parallel ensemble

The underlying classes mapped across both models are 0 = `With Helmet` and 1 = `Without Helmet`.

Fine-tuning details: The primary `helmet_github.pt` model was fine-tuned from pre-trained weights using a comprehensive combined dataset of 7,180 training images and 499 validation images sourced from Roboflow Universe:

- *Bike Helmet Detection* (3,532 images) — `object-detection-using-yolov8`
- *Helmet Detection YOLOv8* (3,648 images) — `learning-evidence`

The training ran for 36 epochs (with early stopping) using a batch size of 8, image resolution of 640 px, an AdamW optimizer, on an NVIDIA RTX 4060 Laptop GPU (8 GB VRAM), reaching an mAP50 of 0.909 and mAP50-95 of 0.585.

Dataset Constraints: Initial fine-tuning passes attempted on native Indian datasets, including the *India Driving Dataset (IDD)*, faced major regressions. These issues stemmed from poor per-rider head label quality, severe label noise, and absent annotations for complex occlusion profiles. Retaining general-purpose fine-tuned weights yielded vastly superior cross-domain generalization.

3.3 Licence Plate Detection and OCR

License plate extraction is executed via a discrete two-stage sub-pipeline: localization followed by Optical Character Recognition.

Plate Localisation: A dedicated **YOLOv11-Nano License Plate Model** (`plate.pt`, size: 5.4 MB) was fine-tuned using localized Indian and international vehicle registration plate datasets compiled from Kaggle and Roboflow Universe. Instead of scanning the entire street-scene image frame, the model operates exclusively on cropped motorcycle bounding boxes generated during Stage 1. Before passing the crop to the plate detector, the bounding boxes are dynamically padded outwards by 15% to ensure no part of front or rear-facing license plates is clipped at the border.

OCR Pipeline: Once a license plate bounding box is isolated, the cropped region is subjected to sequential computer vision preprocessing steps to maximize text readability:

- 4× Bicubic Interpolation Upscaling to increase text pixel density.
- Grayscale Conversion followed by Contrast Limited Adaptive Histogram Equalization (CLAHE) to mitigate harsh outdoor illumination and shadow gradients.
- Bilateral Filtering to reduce sensor and motion noise while firmly preserving crisp character edges.
- Adaptive Thresholding and a Laplacien Sharpening Filter to maximize text-to-background contrast.

Alphanumeric characters are extracted using **PaddleOCR** running entirely locally in an offline runtime environment. Post-extraction string cleaning applies a regular-expression filter (`[^A-Z0-9]`) to eliminate misread punctuation, frame lines, or security rivets.

3.4 Model Size Summary

Model File	Functional Purpose	Model Size
yolo11l.pt	Object Detection (Motorcycles, Persons)	49.0 MB
helmet_github.pt	Head-level Helmet Detection (Primary)	83.6 MB
helmet_iamtsr.pt	Head-level Helmet Detection (Ensemble)	5.9 MB
plate.pt	Licence Plate Localization (Nano)	5.4 MB
Total Memory Footprint		143.9 MB (< 250 MB)

Table 2: System model asset breakdown showing adherence to storage limits

4 Implementation Details

4.1 Rider Association — Hungarian Matching

Associating independent person bounding boxes to their respective two-wheeler vehicles presents a major bottleneck in high-density traffic scenes. Simple greedy distance assignments frequently break when multiple motorcycles travel abreast, causing cross-assignment errors.

We address this by modeling rider association as a minimum-cost bipartite matching problem solved globally via the **Hungarian Algorithm** using `scipy.optimize.linear_sum_assignment`. The entry in the cost matrix mapping a person p to a motorcycle b is formalized as:

$$c(p, b) = \begin{cases} 1 - \text{IoU}(p, b) & \text{if } p \text{ is geometrically eligible for } b \\ 1000 & \text{otherwise} \end{cases}$$

A person p is declared *eligible* for a vehicle b if and only if:

$$\text{IoU}(p, b) \geq 0.15 \tag{1}$$

OR

The horizontal centroid of p falls within the horizontal span of b $[x_{min,b}, x_{max,b}]$ while the bottom boundary of p 's bounding box resides above the lower boundary of b .

Following the global optimization solve, a fallback greedy loop associates any remaining unassigned persons to vehicles matching the vertical stack rules to account for extreme perspective distortions.

4.2 Helmet Matching — Proximity-Weighted Scoring

Rather than slicing individual rider crops—which drops global context—both helmet models evaluate the complete frame simultaneously. The individual helmet predictions are then routed back to the associated riders. For each rider, an upper search envelope is constructed (projecting 50% horizontally and 100% vertically upward relative to the person's bounding box coordinates to capture the head).

For every helmet bounding box found inside this envelope, a **proximity-weighted score** is evaluated:

$$\text{score} = \frac{\text{conf}}{1 + \left(\frac{d}{w_p}\right)}$$

where conf represents the raw confidence score of the model, d is the Euclidean distance from the center of the helmet box to the estimated center of the rider's head, and w_p is the total width of the person's bounding box. This penalizes far-away detections, preventing a high-confidence helmet label belonging to a passing background rider from accidentally resolving an unhelmeted foreground rider violation.

The operational decision boundaries are set as follows:

- **HELMET_CONF = 0.35**: Minimum score boundary to validate a secure *Helmet OK* state.
- **NO_HELMET_MIN_CONF = 0.50**: Minimum score boundary to confirm a definite *No Helmet* infraction.
- Ambiguous boundary cases falling between both cut-offs are assigned to the class reporting the higher raw proximity-weighted index.

4.3 Auto-Rickshaw Filtering

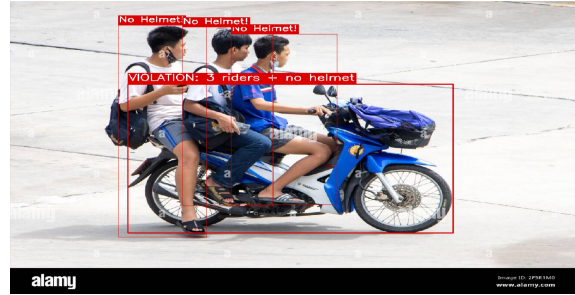
Due to structural similarities in front profiles, three-wheeled auto-rickshaws are periodically misclassified by standard MS COCO weights as motorcycles. The pipeline runs a localized geometry check: if a detected motorcycle bounding box exhibits an aspect ratio (width/height) greater than 1.3 *and* its total box width spans more than 22% of the image's full width, it is categorized as an auto-rickshaw and pruned from the detection stack.

5 Results and Demo

5.1 Working Case 1 — Triple Riding + No Helmet



(a) Input image



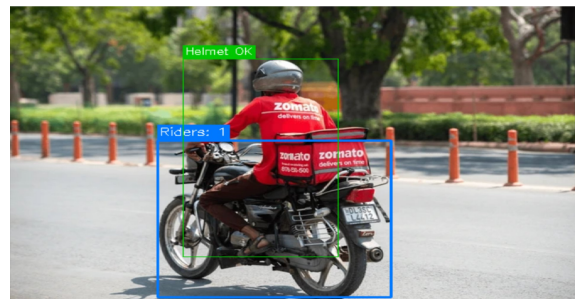
(b) System output

Figure 2: Three riders detected on a single motorcycle, all without helmets. System correctly outputs **VIOLATION: 3 riders + no helmet**.

5.2 Working Case 2 — Helmet Correctly Confirmed



(a) Input image



(b) System output

Figure 3: Single rider correctly identified as wearing a helmet. System outputs **Riders: 1, Helmet OK** with no violation flagged.

5.3 Working Case 3 — Dense Indian Traffic



(a) Input image



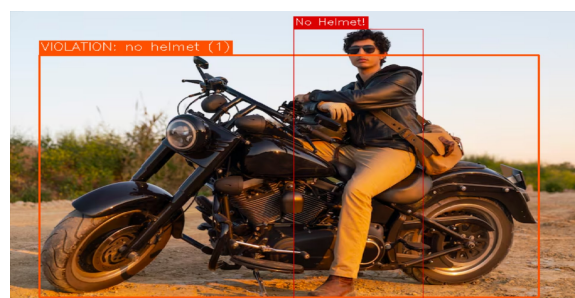
(b) System output

Figure 4: Dense Mumbai traffic scene with multiple scooters. The system correctly detects four riders, all wearing helmets, with no violations flagged. Hungarian matching correctly associates each rider to their respective bike.

5.4 Working Case 4 — No-Helmet Violation (Single Rider)



(a) Input image



(b) System output

Figure 5: Single rider without helmet correctly flagged as **VIOLATION: no helmet (1)**. The system confidently detects the absence of a helmet and raises a violation.

5.5 OCR / Licence Plate Results

The license plate extraction sequence operates with high fidelity under standard working conditions. Figure 6 demonstrates successful localized text region tracking across isolated motorcycles utilizing our system framework.



Figure 6: Licence plate detection and successful OCR analysis case. The sub-network successfully localizes the plate frame boundaries on the two-wheeler vehicle, passing clear character matrices into the offline textual extraction runtime.

6 Failure Cases

6.1 Helmet Status Unknown on Distant Riders

The helmet ensemble frequently fails to produce confident detections for small or distant riders. When no detection falls within a rider’s search zone, the system reports *Helmet:*

Unknown rather than making an incorrect determination.

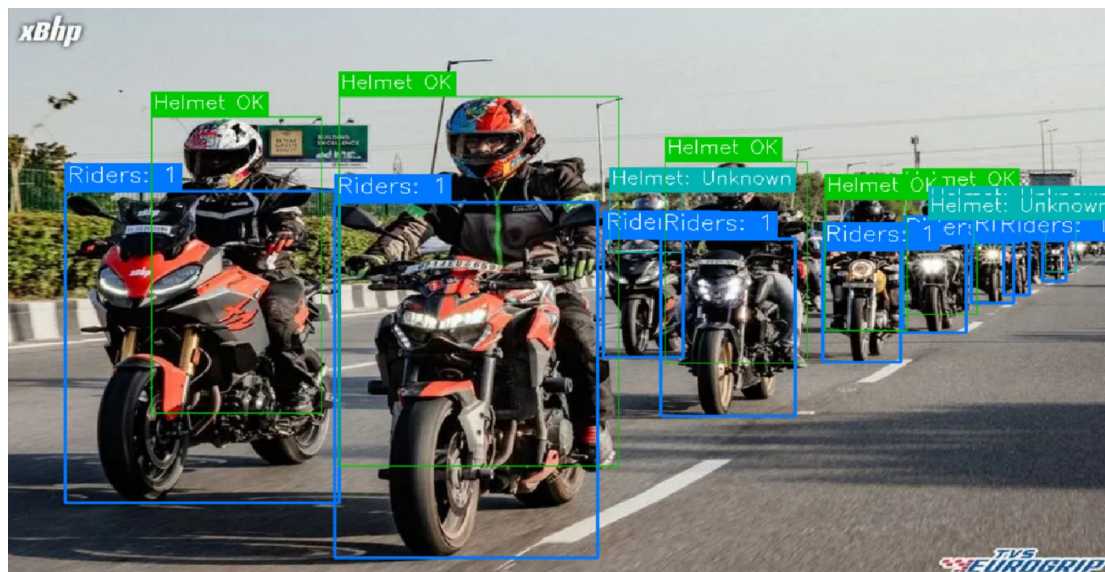


Figure 7: Motorcycle convoy scene. Close-up riders are correctly labelled *Helmet OK*, but distant riders in the background show *Helmet: Unknown* as the model cannot produce confident detections at those scales.

Root cause: The helmet models were trained primarily on close-up images. At small scales (rider head < 30 px), neither model produces a confident detection. Fine-tuning on a more scale-diverse dataset would address this.

6.2 Partially Visible Motorcycles at Image Edges

Bikes partially cut off at image edges are sometimes missed by YOLO, leaving their riders undetected and unassociated with any vehicle.



Figure 8: Aerial view of a Ghaziabad traffic intersection. The motorcycle at the bottom-left is partially cut off and its rider (red helmet) is not associated with any detected bike. Only bikes fully visible in frame are detected and counted correctly.

6.3 Auto-Rickshaw False Positives (Edge Case)

On certain images, auto-rickshaws whose bounding-box aspect ratio falls close to the filter threshold (1.3) are not filtered out and are incorrectly treated as motorcycles. Passengers inside the auto may then be counted as riders.

6.4 OCR Resolution Failure on Motion-Blurred Targets

Character resolution drops sharply under high-speed mobile tracking conditions or severe camera sensor motion blur artifacts. Figure 9 highlights this functional tracking limitation.



Figure 9: Example of tracking-induced OCR processing breakdown. Distant geometries or fast pixel movement paths smooth out character boundaries, causing structural letter mutations or incomplete textual read values inside the deep learning engine.

7 Conclusion and Future Work

We designed and implemented a modular, offline, end-to-end traffic violation detection system that:

- Detects two-wheelers and persons using YOLOv11-Large (49 MB)
- Uses Hungarian matching for globally optimal rider-to-bike association
- Runs a two-model helmet detection ensemble with proximity-weighted scoring to minimise false positives from adjacent riders
- Extracts licence plates using a finetuned detector and PaddleOCR
- Operates entirely offline within the 250 MB model constraint (143.9 MB total usage)

Strengths. The system performs reliably on standard-resolution Indian traffic images. Proximity-weighted helmet scoring significantly reduces false positives. The Hungarian assignment correctly handles multi-bike dense scenes.

Limitations. Helmet detection degrades for distant or small riders due to the scale gap between training data and real Indian traffic imagery. The auto-rickshaw filter has edge cases near its threshold, and heavy motion blur degrades raw OCR string accuracy.

Future Work. Incorporating tracking algorithms like DeepSORT or ByteTrack will allow the system to average OCR predictions over multiple video frames to counter instantaneous motion blur. Training a dedicated small-scale helmet detector and utilizing depth estimation to weight rider assignments would further improve real-world street deployment.

References

- [1] Ultralytics YOLOv11. <https://github.com/ultralytics/ultralytics>
- [2] PaddleOCR. <https://github.com/PaddlePaddle/PaddleOCR>
- [3] Helmet Detection Model (Primary). <https://github.com/Juliowiwiiwi/Bike-Helmet-Detection-Model>
- [4] YOLOv8 Helmet Detection Model (Ensemble). <https://huggingface.co/iam-tsr/yolov8n-helmet-detection>